

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming (Coding Challenge)

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO2-AT2-CODING CHALLENGE
Question	Q1 — Challenge 1: String Character Frequency Counter (Easy 10 Marks)

Challenge 1: String Character Frequency Counter [Level: Easy | 10 Marks]

Write a Python program that accepts a string from the user and counts the frequency of each unique character (case-insensitive). Display the results in the format: character → count. Sort the output alphabetically by character.

Constraints

- Ignore spaces in the count.
- Treat uppercase and lowercase as the same character.
- Sort output alphabetically.

Skills Assessed

- String iteration, lower(), conditionals
- Dictionary or manual counting
- Sorting using loops or sorted()

Sample Input / Output

Sample Input:
Hello World

Expected Output:
d -> 1
e -> 1
h -> 1
l -> 3
o -> 2
r -> 1
w -> 1

Approach / Design

- **Counting (count_character_frequency):** iterates through the input string one character at a time, skips spaces with a conditional check, lowercases each character so case is treated uniformly, and tallies counts manually in a dictionary.
- **Sorting (sort_frequency_alphabetically):** uses sorted() on the dictionary's (character, count) items, which sorts alphabetically by character since that's the first element of each tuple.
- **Display (display_frequency):** loops through the sorted items and prints each in the required 'character -> count' format.
- **Main driver (main):** reads the string from the user via input(), then chains the counting, sorting, and display functions together.

Python Program

```
"""
C02-AT2 - Coding Challenge
Challenge 1: String Character Frequency Counter [Level: Easy | 10 Marks]
-----
Accepts a string from the user and counts the frequency of each
unique character (case-insensitive), ignoring spaces. Displays
the results as "character -> count", sorted alphabetically.
"""

# -----
# 1. Frequency counting function
# -----
def count_character_frequency(text):
    """
    Iterates through the input string, normalizes case using
    lower(), ignores spaces, and counts occurrences of each
    character using a dictionary (manual counting).
    """
    frequency = {}

    for char in text:
        if char == " ":
            continue # ignore spaces in the count

        char = char.lower() # treat uppercase and lowercase as the same

        if char in frequency:
            frequency[char] += 1
        else:
            frequency[char] = 1

    return frequency

# -----
# 2. Sorting function
# -----
def sort_frequency_alphabetically(frequency):
    """
    Returns the frequency dictionary's items sorted alphabetically
    by character, using sorted() on the keys.
    """
    return sorted(frequency.items())

# -----
```

```

# 3. Display function
# -----
def display_frequency(sorted_items):
    """
    Prints each character and its count in the format:
    character -> count
    """
    for char, count in sorted_items:
        print(f"{char} -> {count}")

# -----
# 4. Main driver
# -----
def main():
    text = input("Enter a string: ")

    frequency = count_character_frequency(text)
    sorted_items = sort_frequency_alphabetically(frequency)

    print("\nCharacter Frequency (alphabetical order):")
    display_frequency(sorted_items)

if __name__ == "__main__":
    main()

```

Sample Output

```
Enter a string: Hello World
```

```
Character Frequency (alphabetical order):
```

```
d -> 1
```

```
e -> 1
```

```
h -> 1
```

```
l -> 3
```

```
o -> 2
```

```
r -> 1
```

```
w -> 1
```

Conclusion

The program counts character frequencies in a single pass over the input string, using a conditional to skip spaces and `lower()` to normalize case so that, for example, 'H' and 'h' are counted together. Results are sorted alphabetically with `sorted()` and displayed in the required 'character -> count' format. Running the program on the sample input "Hello World" reproduces the expected output exactly.